



Domain-Driven Design (DDD) e Arquitetura de Software — TP1

Nome: Guilherme de Morais China Costa

Data: 09/08/2026

Parte 1 — Fundamentos e Arquitetura

1. Explique de forma sucinta o que são microsserviços.

Microsserviços são uma forma de dividir um sistema em aplicações menores, cada uma responsável por uma parte do negócio. Em um e-commerce, por exemplo, pagamento, estoque e entrega podem ser serviços separados. Eles conversam entre si por APIs ou mensagens, mas cada um pode ser desenvolvido e publicado no seu próprio ritmo.

2. Cite uma vantagem da arquitetura de microsserviços.

Uma vantagem é escalar só o que realmente precisa. Em uma promoção, o pagamento pode receber muito mais acessos que o cadastro de usuários. Com microsserviços, é possível aumentar a capacidade apenas do serviço de pagamento, sem gastar recursos com o sistema inteiro.

3. Cite uma desvantagem da arquitetura de microsserviços.

O ganho de independência traz mais trabalho operacional. Em vez de acompanhar uma aplicação, a equipe passa a cuidar de vários serviços, integrações, logs e configurações. Além disso, uma comunicação que antes era uma chamada interna pode falhar por causa da rede.

4. Cite uma característica dos microsserviços que os diferenciam de outras arquiteturas de software.

A principal característica é a autonomia de implantação. Um serviço pode receber uma atualização sem que todos os demais sejam publicados junto com ele. Para isso funcionar bem, os contratos de comunicação precisam ser estáveis e respeitados pelos serviços envolvidos.

5. Explique de forma sucinta o que é um monólito.

Um monólito é uma aplicação em que as funcionalidades ficam reunidas no mesmo projeto e, em geral, são publicadas de uma vez. Ele pode ter uma boa divisão interna de módulos, mas uma alteração no sistema normalmente gera uma nova versão da aplicação inteira.

6. Explique de forma sucinta o que significa Acoplamento do ponto de vista de Engenharia de Software.

Acoplamento é o grau de dependência entre partes do software. Se o módulo de pagamento precisa conhecer tabelas, classes ou regras internas do estoque para funcionar, o acoplamento é alto. Quanto menor essa dependência, mais fácil fica alterar, testar e reaproveitar cada parte sem gerar efeitos inesperados.

7. Explique de forma sucinta o que significa Coesão do ponto de vista de Engenharia de Software.

Coesão indica se as responsabilidades de uma classe ou serviço fazem sentido juntas. Um serviço de pagamento com regras de cobrança, aprovação e estorno tem boa coesão. Se ele também começar a cuidar do cadastro de produtos, passa a reunir assuntos diferentes e fica mais difícil de manter.

8. Explique de forma sucinta o que é um Agregado do DDD.

No DDD, um agregado é um grupo de objetos do domínio que deve permanecer consistente. Ele tem uma porta de entrada, a Aggregate Root, que concentra as mudanças importantes. Assim, em vez de qualquer classe alterar livremente os dados, a raiz garante que as regras do negócio sejam seguidas.

Parte 1 — Fundamentos e Arquitetura

9. Cite uma vantagem de construir microsserviços usando Agregados do DDD.

Os agregados ajudam a encontrar uma fronteira clara para cada serviço. No caso de pagamento, as regras para aprovar ou recusar uma cobrança ficam no próprio contexto de Pagamento. Isso evita que detalhes de estoque ou usuários se espalhem pelo código e reduz a dependência entre serviços.

10. Dê um exemplo de mapeamento de um Contexto Delimitado para um microserviço, utilizando a linguagem Java.

O Contexto Delimitado de Pagamentos pode se tornar o microserviço `pagamento-service`. Nele ficam classes como `Pagamento`, `ProcessarPagamento` e os adaptadores do gateway de pagamento. Esse serviço entende de cobranças e confirmações, mas não precisa saber como o Estoque ou o Usuário são implementados internamente.

```
br.edu.infnet.pagamento
├── domain           // regras: Pagamento, StatusPagamento
├── application      // casos de uso: ProcessarPagamento
└── infrastructure  // adaptadores: ClienteMonolitoHttp, gateway
```

11. Dê um exemplo de mapeamento de um Agregado, com Objeto de Valor, para o contexto de um microserviço, utilizando a linguagem Java.

No serviço de Pagamento, a classe `Pagamento` é a raiz do agregado, pois é ela que decide quando o status pode mudar de pendente para aprovado ou recusado. Já `Dinheiro` é um Objeto de Valor: interessa o valor e a moeda que ele representa, e não uma identidade própria. Essa modelagem aparece no código da questão 14.

12. Porque o compartilhamento de banco de dados é uma estratégia de integração ruim sob o ponto de vista de microsserviços?

Quando vários serviços usam o mesmo banco, eles continuam presos uns aos outros. Uma alteração de tabela, uma consulta lenta ou um bloqueio pode afetar mais de um serviço. Também fica difícil saber quem é responsável por cada dado. Por isso, o mais saudável é cada serviço ser dono de sua base e trocar informações por API ou eventos.

Parte 2 — Estudo de Caso e Refatoração Prática

Cenário: sistema monolítico de e-commerce legado, com Usuário, Estoque e Pagamento compartilhando banco de dados e classes fortemente acopladas. A migração para microsserviços deve ocorrer gradualmente.

13. Explique qual estratégia de migração você utilizaria para iniciar a transformação desse monólito em microsserviços de forma segura, sem precisar desligar o sistema atual.

Eu usaria o padrão Strangler Fig, também chamado de Padrão Estrangulador. Primeiro, criaria uma entrada única para as operações de pagamento. Essa entrada encaminharia uma parte controlada das requisições ao novo serviço, enquanto o monólito continuaria atendendo o restante. Depois de acompanhar os resultados e corrigir o que fosse necessário, novos fluxos seriam direcionados ao microsserviço até o caminho antigo deixar de ser necessário. Dessa forma, a empresa evita uma troca brusca e consegue recuar se surgir algum problema.

Questão 14 — Desafio de Implementação

14. O seu primeiro passo será extrair o contexto de Pagamento. Aplique os princípios de DDD e o padrão de migração escolhido para implementar a estrutura inicial desse novo microserviço em Java. Insira os blocos de código demonstrando a classe do novo Aggregate Root de Pagamento e a comunicação com o monólito legado, garantindo baixo acoplamento.

Foi criado um projeto Java com Maven chamado `pagamento-service`. A classe `Pagamento` concentra a regra de aprovação, enquanto `Dinheiro` representa o valor cobrado e sua moeda. Para o novo serviço não depender de classes ou tabelas do monólito, a comunicação acontece por uma interface chamada `NotificadorMonolito`. O adaptador HTTP é o único ponto que conhece o sistema legado, o que facilita trocar a integração no futuro.

Aggregate Root e Objeto de Valor

```
public final class Pagamento {
    private final UUID id = UUID.randomUUID();
    private final UUID pedidoId;
    private final Dinheiro valor;
    private StatusPagamento status = StatusPagamento.PENDENTE;

    public Pagamento(UUID pedidoId, Dinheiro valor) {
        this.pedidoId = Objects.requireNonNull(pedidoId);
        this.valor = Objects.requireNonNull(valor);
    }

    public void aprovar() {
        if (status != StatusPagamento.PENDENTE)
            throw new IllegalStateException("Pagamento já processado");
        status = StatusPagamento.APROVADO;
    }

    public record Dinheiro(BigDecimal valor, String moeda) {
        public Dinheiro {
            if (valor == null || valor.signum() <= 0)
                throw new IllegalArgumentException("Valor deve ser positivo");
        }
    }
}
```

Porta de integração e adaptador para o legado

```
public interface NotificadorMonolito {
    void pagamentoProcessado(UUID pedidoId, UUID pagamentoId,
                             Pagamento.StatusPagamento status);
}

public final class ClienteMonolitoHttp implements NotificadorMonolito {
    @Override
    public void pagamentoProcessado(UUID pedidoId, UUID pagamentoId,
                                     Pagamento.StatusPagamento status) {
        // POST /integracoes/pedidos/{pedidoId}/pagamento
        // Sem acesso às tabelas nem às classes internas do monólito.
    }
}
```

Com essa separação, as regras de pagamento ficam concentradas no agregado e a comunicação com o legado fica isolada no adaptador. Durante a migração, a camada de entrada do sistema pode enviar os pagamentos ao novo serviço de forma gradual. Se alguma etapa ainda não estiver pronta, o fluxo antigo pode continuar ativo temporariamente, sem misturar os dois códigos.